

Learning and Reusing Policy Decompositions for Hierarchical Generalized Planning with LLM Agents

Shirin Sohrabi, Haritha Ananthakrishnan, Harsha Kokel,
Kavitha Srinivas, Michael Katz
IBM

**LLM agents should learn reusable policy components,
not solve every task from scratch**

Why reuse?

Observation: repeated structure is common, but most agents restart from scratch

- ▶ Interactive coding agents generate executable code, call APIs, inspect feedback, and repair failures
- ▶ But similar tasks often share the same abstract flow
- ▶ Without reuse:
 - ▶ near-duplicate reasoning is repeated
 - ▶ inference budget is spent rediscovering procedures
 - ▶ **successful** procedural knowledge is not accumulated

Planning perspective: learn generalized policies and reusable decompositions

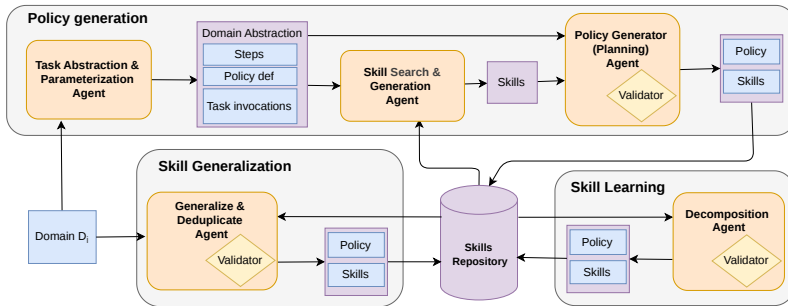
Informal formalism – What are we solving?

Domains are collections of related task instances

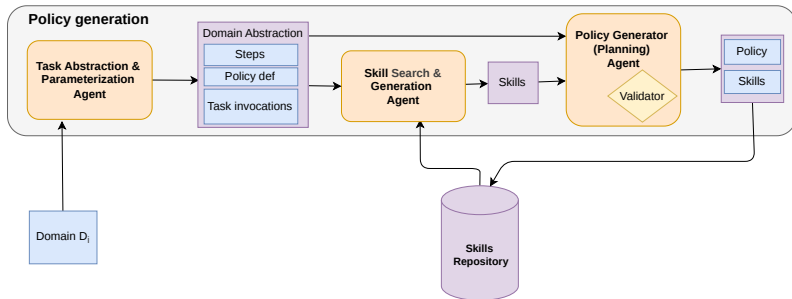
- ▶ Task instance: natural-language instruction, initial state, goal specification
- ▶ Domain: tasks share procedural structure, differ mostly in parameters
- ▶ Policy: parameterized executable program
- ▶ To solve a task:
 - ▶ extract parameters
 - ▶ instantiate the policy
 - ▶ execute
 - ▶ validate

Task instances → Parameterized policy → Validated executions

Hierarchical Component Learning for Generalized Policies HCL-GP



Hierarchical Component Learning for Generalized Policies HCL-GP

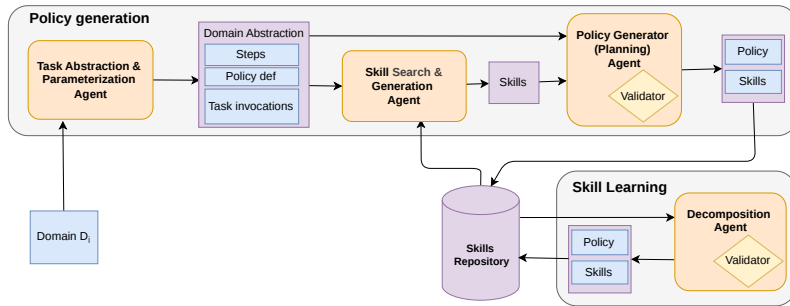


Three stages, each guarded by validation

- **1. Generate policy**

- abstract tasks, infer signature, bind parameters, retrieve skills, synthesize code

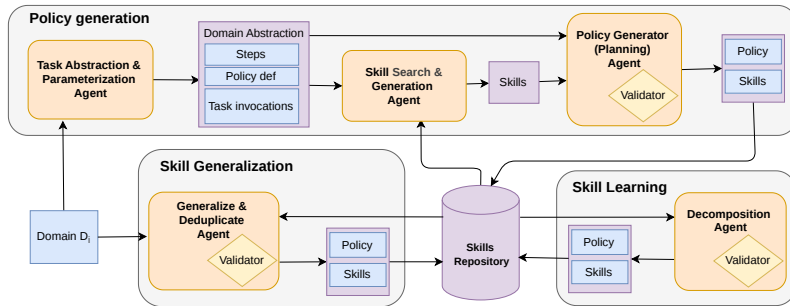
Hierarchical Component Learning for Generalized Policies HCL-GP



Three stages, each guarded by validation

- ▶ **1. Generate policy**
 - ▶ abstract tasks, infer signature, bind parameters, retrieve skills, synthesize code
- ▶ **2. Learn skills**
 - ▶ decompose successful policies into reusable functions and revalidate updated policies

Hierarchical Component Learning for Generalized Policies HCL-GP



Three stages, each guarded by validation

- ▶ **1. Generate policy**
 - ▶ abstract tasks, infer signature, bind parameters, retrieve skills, synthesize code
- ▶ **2. Learn skills**
 - ▶ decompose successful policies into reusable functions and revalidate updated policies
- ▶ **3. Generalize skills**
 - ▶ cluster, deduplicate, abstract, and validate before adding to the repository

Policy Generation

Inputs to the Policy Generator Agent

- ▶ High-level workflow steps induced from the domain tasks
- ▶ Policy signature: the interface of the generalized policy
- ▶ Concrete parameter bindings for each training task instance
- ▶ Relevant reusable skills from the validated repository

Validation and feedback

- ▶ Generated policy is instantiated and executed for all train tasks
- ▶ Failures return traces and error messages for iterative repair

Skill Learning, Generalization, and Reuse

Skills are coherent executable subprocedures

- ▶ Extracted only after a domain-level policy succeeds
- ▶ Similar skills can be generalized, e.g., app-specific login $\rightarrow$ login_to_app
- ▶ Updated policy must still validate on the original training tasks

AppWorld instantiation

A natural meta-domain for generalized, executable policies

- ▶ Simulated applications exposed through APIs
- ▶ World state stored in application databases
- ▶ Agent cannot directly edit state, only issue API calls
- ▶ Each scenario is treated as a domain with multiple related tasks
- ▶ Normal test set evaluates in-distribution generalization
- ▶ Challenge test set includes unseen applications such as Gmail and Amazon
- ▶ Metrics: TGC for task success, SGC for all-tasks-in-scenario success



Experimental setup

Compare to existing results

- ▶ Recent literature
- ▶ AppWorld leaderboard

Our Setup

- ▶ Two models: GPT OSS 120B (open-source) and Claude Sonnet 4.6
- ▶ Run on training set, obtain the reusable skills (with Claude Sonnet 4.6)
- ▶ Seed with the learned reusable skills
- ▶ Baseline: no skills, no skills learning

Results on AppWorld

	Method	Model	Normal		Challenge	
			TGC	SGC	TGC	SGC
Bandel et al. (2026)	Claude Code	Claude Opus 4.5	66.0	56.6	—	—
	Smolagents Code	Claude Opus 4.5	70.0	60.4	—	—
	OpenAI Solo	Claude Opus 4.5	68.0	56.6	—	—
	ReAct	Claude Opus 4.5	61.0	52.8	—	—
	ReAct Short	Claude Opus 4.5	64.0	54.7	—	—

Results on AppWorld

	Method	Model	Normal		Challenge	
			TGC	SGC	TGC	SGC
Bandel et al. (2026)	Claude Code	Claude Opus 4.5	66.0	56.6	—	—
	Smolagents Code	Claude Opus 4.5	70.0	60.4	—	—
	OpenAI Solo	Claude Opus 4.5	68.0	56.6	—	—
	ReAct	Claude Opus 4.5	61.0	52.8	—	—
	ReAct Short	Claude Opus 4.5	64.0	54.7	—	—
AppWorld Leaderboard	Alibaba AgentRL	Qwen3-14B	86.9	80.4	67.6	50.4
	IBM CUGA	GPT-4.1	73.2	62.5	57.6	48.2
	LOOP	Qwen2.5-32B	72.6	53.6	47.2	28.8

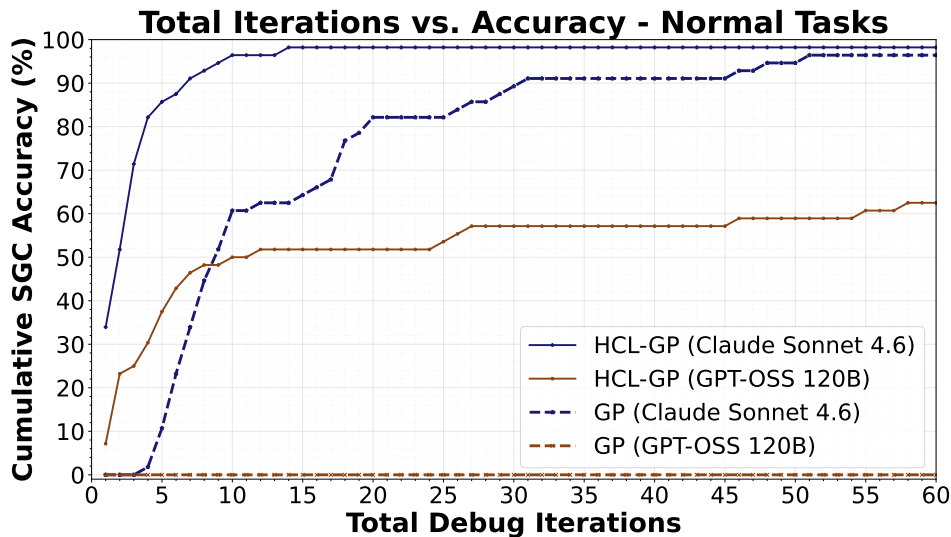
Results on AppWorld

	Method	Model	Normal		Challenge	
			TGC	SGC	TGC	SGC
Bandel et al. (2026)	Claude Code	Claude Opus 4.5	66.0	56.6	—	—
	Smolagents Code	Claude Opus 4.5	70.0	60.4	—	—
	OpenAI Solo	Claude Opus 4.5	68.0	56.6	—	—
	ReAct	Claude Opus 4.5	61.0	52.8	—	—
	ReAct Short	Claude Opus 4.5	64.0	54.7	—	—
AppWorld Leaderboard	Alibaba AgentRL	Qwen3-14B	86.9	80.4	67.6	50.4
	IBM CUGA	GPT-4.1	73.2	62.5	57.6	48.2
	LOOP	Qwen2.5-32B	72.6	53.6	47.2	28.8
	GP (baseline)	GPT-OSS 120B	0.0	0.0	0.7	0.7
		Claude Sonnet 4.6	96.4	96.4	83.2	82.0

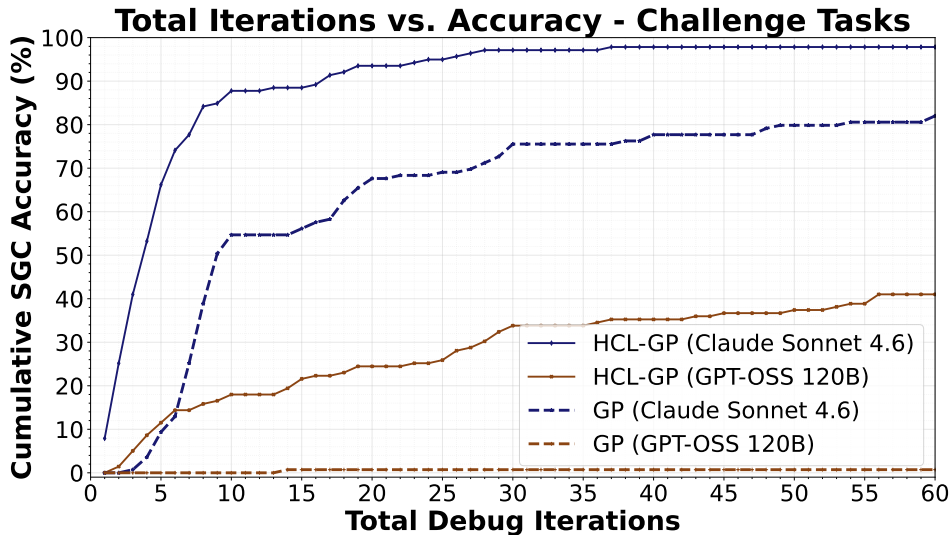
Results on AppWorld

	Method	Model	Normal		Challenge	
			TGC	SGC	TGC	SGC
Bandel et al. (2026)	Claude Code	Claude Opus 4.5	66.0	56.6	—	—
	Smolagents Code	Claude Opus 4.5	70.0	60.4	—	—
	OpenAI Solo	Claude Opus 4.5	68.0	56.6	—	—
	ReAct	Claude Opus 4.5	61.0	52.8	—	—
	ReAct Short	Claude Opus 4.5	64.0	54.7	—	—
AppWorld Leaderboard	Alibaba AgentRL	Qwen3-14B	86.9	80.4	67.6	50.4
	IBM CUGA	GPT-4.1	73.2	62.5	57.6	48.2
	LOOP	Qwen2.5-32B	72.6	53.6	47.2	28.8
	GP (baseline)	GPT-OSS 120B	0.0	0.0	0.7	0.7
		Claude Sonnet 4.6	96.4	96.4	83.2	82.0
	HCL-GP	GPT-OSS 120B	62.5	62.5	41.0	41.0
		Claude Sonnet 4.6	98.2	98.2	98.3	97.8

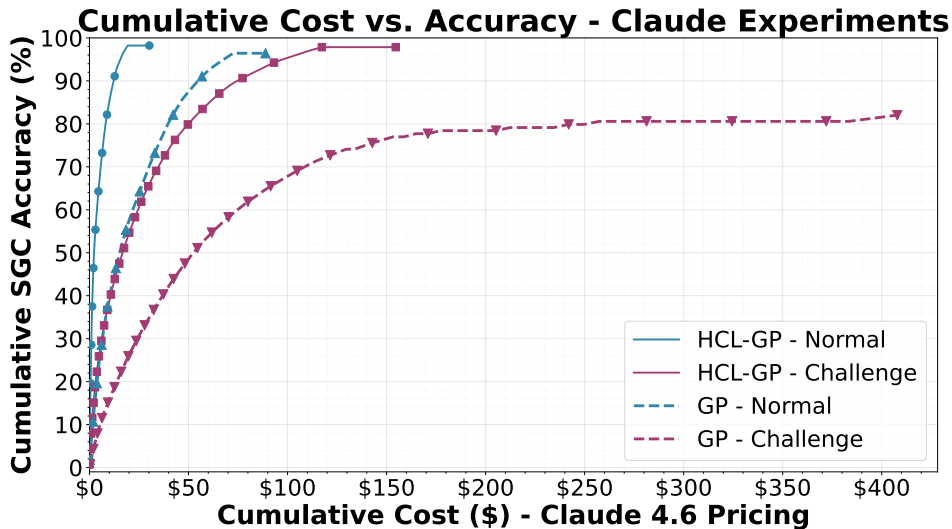
Closer Look: Iteration Efficiency



Closer Look: Iteration Efficiency



Closer Look: Cost



Conclusions

Planning structure remains useful, even when policies are generated by LLMs

- ▶ Reuse improves transfer to unseen applications and helps weaker models
- ▶ Validation is the correctness signal that makes reuse safe
- ▶ HTN connection: from partial order to glue code

LLM agents should accumulate, generalize, and reuse planning knowledge